ChatterBox Project – Technical Interview Questions & Answers

A. Project Overview

- What is ChatterBox? – A real-time chat app built with MERN, Socket.IO, Zustand, and Cloudinary.
- What makes your project unique? – Real-time updates, modular backend, and secure JWT auth.
- Motivation – To understand real-time communication and scalable backend systems.

B. Frontend & State Management

- Why React? – For fast, component-based UI updates.
- Why Zustand instead of Redux? – Lightweight, simple, and ideal for socket & auth state.
- How do you handle real-time UI updates? – Using socket events (socket.on) and Zustand store.

C. Backend & Database

- Why Express.js? – Lightweight and integrates easily with middlewares like CORS, cookie-parser.
- Why MongoDB? – Flexible schema for unstructured chat data and fast queries.
- How is authentication implemented? – Using JWT tokens stored in secure HTTP-only cookies.
- How are passwords secured? – Hashed using bcryptjs before saving to MongoDB.

D. Real-Time Functionality

- What is Socket.IO? – A library for real-time, event-driven communication over WebSockets.
- Difference between io.emit() and socket.emit()? – io.emit sends to all; socket.emit sends to one client.
- How are online users tracked? – Using a userSocketMap storing userId–socketId pairs.
- How is new message delivery handled? – io.to(receiverSocketId).emit('newMessage', message).

E. File Uploads & Cloud Storage

- Why Multer? – To handle image/file uploads from frontend forms.
- Why Cloudinary? – Cloud-based image storage, easy URL access, and automatic optimization.

F. Challenges & Problem Solving

- Main challenges – Managing socket reconnections, real-time sync, and auth persistence.
- How were sync issues resolved? – Using socket.on('disconnect') and re-emitting updated user lists.
- Debugging approach – Console logs for socket connections and message flow testing.

G. Deployment & Scalability

- Where did you deploy? – Frontend: Vercel; Backend: Render; DB: MongoDB Atlas.
- How is CORS handled? – app.use(cors({ origin: 'http://localhost:5173', credentials: true })).
- How would you scale it? – Use socket rooms, horizontal scaling, and load balancing.

H. Koch Industries Focus Questions

- What difficulties did you face? – Synchronization, auth persistence, handling file uploads.
- Why MongoDB over SQL? – Flexible schema, faster inserts, scalable for real-time data.
- Explain JWT Authentication. – Token-based login system without storing sessions on the server.
- What are middlewares? – Functions that run before route handlers to verify/authenticate.